The Chip

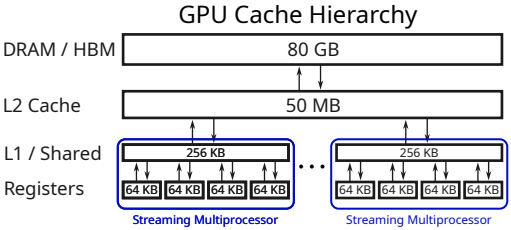
🤘 Cache Me If You Can, Talk 2

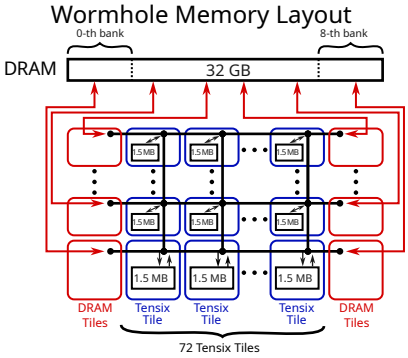
Maciek Dragula Nathan Maurice

Tenstorrent

October 11, 2026

Differences between Tenstorrent accelerators and GPUs





Unlike GPUs, Tenstorrent chips have:

- *Lots of* SRAM across a grid of cores
- Flexible data movement, but managed manually
- No graphics (textures, video decoders)
- No HBM (expensive & hard to get)

What runs our code?

Operations run on a Tenstorrent **Wormhole N150** card.

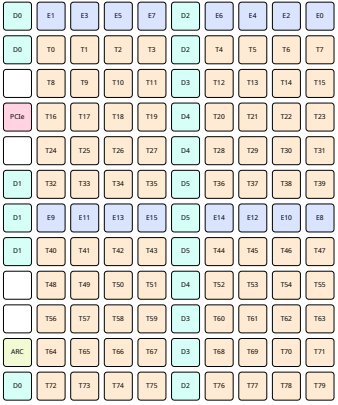
The N150 has:

- A grid of 80 **Tensix cores**
- 12 GB of DRAM, at $\approx$ 288 GB/s.
- External Ethernet connections

On each Tensix core:

- 1.5 MB of fast SRAM
- Matrix and vector engines

Cores can talk to DRAM and to each other over a **Network-on-Chip** (NoC).



Inference is all about data movement

DRAM is **far** and comparatively slow to reach, but we have lots of it.

- This is where we want our KV-cache data.

SRAM is **in each core** and is **near**. It is very fast, but available in small amounts.

- This is where calculations happen.

In **Decode**, we move weights from **DRAM** (far) to **SRAM** (near).

- Weights are read, used for one operation, and then discarded at the end of the step.

We move *lots* of data all the time:

- How fast data moves determines model speed, **regardless** of compute speed.

How software runs on the hardware

Every operation runs a small program on the cores, called a **kernel**.

- Examples are matrix multiplication, activation, reshaping, and softmax.

A kernel must be compiled beforehand. This can be expensive.

- To avoid repeated compilation, kernels are compiled only on the first run.

The **warm-up** round in Act 0 ran all kernels:

- We won't need to compile them again.

Minimizing cost of kernel dispatch

Running one decode step requires calling hundreds of operations.

- Examples: additions, matrix multiplications, and softmax.

Operations are sent as **commands** from the host CPU to the device. This is **slow**.

Idea: Record commands in a **trace**, then replay them for subsequent tokens.

We enable this for models in Acts I, II, and IV.
In Act IV, you will use trace yourself.

What Act I measured

Time to first token grew with prompt length: prefill processes every prompt token.

Tokens per second barely moved:

- The cost of decode is mostly reading the weights.

The gap between measurements and the speed limit is time spent on work other than reading weights.

Act II

1. Implement attention yourself on a real GPT-2 model.
2. Generate text **without** a KV cache: for every token, all intermediate tensors will be recalculated.
3. Add a KV cache. With the same text, we should see a different cost.